

杂题选讲

中山纪念中学 全柏锋

2026.5.22

前言

本来想做 ds 选讲的，但是因为大家太卷了，很多题都有不少人做过，所以就做成普通的杂题选讲了。

[PKUCPC2026] Reborn in death as the cherry blossoms fall

[PKUCPC2026] Reborn in death as the cherry blossoms fall

这是一道交互题。

给定一棵树，你需要通过不超过 35 次询问一个点集的导出子图的连通块数量，找到这棵树的一个叶子。

数据范围： $n \leq 2 \times 10^5$ 。

Hint

Hint

Hint 1: $\sum_{i=1}^n \deg_i < 2n$ 。

Hint

Hint 1: $\sum_{i=1}^n \deg_i < 2n$ 。

Hint 2: 考虑减治。

Hint

Hint 1: $\sum_{i=1}^n \deg_i < 2n$ 。

Hint 2: 考虑减治。

Hint 3: 可以通过一次询问知道一个点是否为叶子。

Solution

Solution

设 $U = \{1, 2, \dots, n\}$, 则初始时 $\sum_{x \in S} \deg_x = 2|S| - 2$ 。

Solution

设 $U = \{1, 2, \dots, n\}$, 则初始时 $\sum_{x \in S} \deg_x = 2|S| - 2$ 。

考虑把 U 划分为两个集合 L, R , 则 $\sum_{x \in L} \deg_x < 2|L|$ 和 $\sum_{x \in R} \deg_x < 2|R|$ 至少有一个满足。

Solution

设 $U = \{1, 2, \dots, n\}$, 则初始时 $\sum_{x \in S} \deg_x = 2|S| - 2$ 。

考虑把 U 划分为两个集合 L, R , 则 $\sum_{x \in L} \deg_x < 2|L|$ 和 $\sum_{x \in R} \deg_x < 2|R|$ 至少有一个满足。

每次把 U 变成满足限制的那个子集, 最后就可以得到一个叶子。

Solution

设 $U = \{1, 2, \dots, n\}$, 则初始时 $\sum_{x \in S} \deg_x = 2|S| - 2$ 。

考虑把 U 划分为两个集合 L, R , 则 $\sum_{x \in L} \deg_x < 2|L|$ 和 $\sum_{x \in R} \deg_x < 2|R|$ 至少有一个满足。

每次把 U 变成满足限制的那个子集, 最后就可以得到一个叶子。

现在问题变为给定点集 S , 求出 $\sum_{x \in S} \deg_x$ 。

Solution

Solution

令 T 为 S 的补，我们把所有边划分成三类：

Solution

令 T 为 S 的补，我们把所有边划分成三类：

- S 内部的边。

Solution

令 T 为 S 的补，我们把所有边划分成三类：

- S 内部的边。
- T 内部的边。

Solution

令 T 为 S 的补，我们把所有边划分成三类：

- S 内部的边。
- T 内部的边。
- $S - T$ 之间的边。

Solution

令 T 为 S 的补，我们把所有边划分成三类：

- S 内部的边。
- T 内部的边。
- $S - T$ 之间的边。

则只需求出这三种边的数量即可得到 $\sum_{x \in S} \deg_x$ 。

Solution

令 T 为 S 的补，我们把所有边划分成三类：

- S 内部的边。
- T 内部的边。
- $S - T$ 之间的边。

则只需求出这三种边的数量即可得到 $\sum_{x \in S} \deg_x$ 。

前两种边的数量可以通过点减边容斥得到，又因为三种边数量和为 $n - 1$ ，所以第三种边的数量也可以求得。

Solution

令 T 为 S 的补，我们把所有边划分成三类：

- S 内部的边。
- T 内部的边。
- $S - T$ 之间的边。

则只需求出这三种边的数量即可得到 $\sum_{x \in S} \deg_x$ 。

前两种边的数量可以通过点减边容斥得到，又因为三种边数量和为 $n - 1$ ，所以第三种边的数量也可以求得。

这样交互次数是 $2^{\lceil \log_2 n \rceil} = 36$ ，注意到当 $U = \{x, y\}$ 时，可以通过询问 $\{x\}$ 的补集的连通块数量得到 x 是否为叶子，从而减少一次交互次数。

[COI 2025] 贪腐

[COI 2025] 贪腐

给定 n 和 $a_0, a_1, \dots, a_{n-1}$, 要求将 $0, 1, \dots, 2^n - 1$ 两两匹配, 使得每个匹配中元素的异或和是 2 的幂, 且异或和是 2^k 的匹配对数量恰好为 a_k 。

数据范围: $n \leq 20$ 。

Hint

Hint

Hint 1: a 全是偶数是合法的必要条件。

Hint

Hint 1: a 全是偶数是合法的必要条件。

Hint 2: 这个条件是充分的。

Hint

Hint 1: a 全是偶数是合法的必要条件。

Hint 2: 这个条件是充分的。

Hint 3: 递归构造。

Hint

Hint 1: a 全是偶数是合法的必要条件。

Hint 2: 这个条件是充分的。

Hint 3: 递归构造。

Hint 4: 每一位是独立的。

Solution

Solution

首先容易找到一个必要条件：所有 a 都是偶数，因为 $0 \sim 2^n - 1$ 的异或和是 0。

Solution

首先容易找到一个必要条件：所有 a 都是偶数，因为 $0 \sim 2^n - 1$ 的异或和是 0。

我们大胆猜测这是充要，并通过递归构造。

Solution

首先容易找到一个必要条件：所有 a 都是偶数，因为 $0 \sim 2^n - 1$ 的异或和是 0。

我们大胆猜测这是充要，并通过递归构造。

假设我们得到了一个 $n - 1$ 的构造，我们可以先把它复制一份（复制品的最高位为 1），得到一个 n 的构造。

Solution

首先容易找到一个必要条件：所有 a 都是偶数，因为 $0 \sim 2^n - 1$ 的异或和是 0。

我们大胆猜测这是充要，并通过递归构造。

假设我们得到了一个 $n - 1$ 的构造，我们可以先把它复制一份（复制品的最高位为 1），得到一个 n 的构造。

设其中有一对匹配 $x \oplus y = 2^k$ ，则我们可以把 $(x, y), (x \oplus 2^{n-1}, y \oplus 2^{n-1})$ 调整成 $(x, x \oplus 2^{n-1}), (y, y \oplus 2^{n-1})$ ，实现一个 $a_k \leftarrow a_k - 2, a_{n-1} \leftarrow a_{n-1} + 2$ 的过程。

Solution

首先容易找到一个必要条件：所有 a 都是偶数，因为 $0 \sim 2^n - 1$ 的异或和是 0。

我们大胆猜测这是充要，并通过递归构造。

假设我们得到了一个 $n - 1$ 的构造，我们可以先把它复制一份（复制品的最高位为 1），得到一个 n 的构造。

设其中有一对匹配 $x \oplus y = 2^k$ ，则我们可以把 $(x, y), (x \oplus 2^{n-1}, y \oplus 2^{n-1})$ 调整成 $(x, x \oplus 2^{n-1}), (y, y \oplus 2^{n-1})$ ，实现一个 $a_k \leftarrow a_k - 2, a_{n-1} \leftarrow a_{n-1} + 2$ 的过程。

因此，我们只要能把 a_{n-1} 分给 $a_0, a_1, \dots, a_{n-2}$ ，使得它们都是 4 的倍数，就能递归构造了。

Solution

Solution

但是可能会存在不够分的情况，注意到每一位是独立的，因此可以将最大的 a 和 a_{n-1} 交换，此时必有 $a_{n-1} \geq \lceil \frac{2^{n-1}}{n} \rceil$ ，只要这个数比 $2(n-1)$ 大，就必定能调整。

Solution

但是可能会存在不够分的情况，注意到每一位是独立的，因此可以将最大的 a 和 a_{n-1} 交换，此时必有 $a_{n-1} \geq \lceil \frac{2^{n-1}}{n} \rceil$ ，只要这个数比 $2(n-1)$ 大，就必定能调整。

不能调整时必有 $n \leq 6$ ($n = 7$ 时 $a_{n-1} \geq 10$ ，只剩下 6 个 a 都 $\text{mod} 4 = 2$ 时才不合法，但这样 a 的和就不是 4 的倍数了，矛盾)，此时直接爆搜即可。

[JOISC 2020] 变色龙之恋

[JOISC 2020] 变色龙之恋

这是一道交互题。有 $2n$ 只变色龙，其中 n 只的性别为 X ，另外 n 只的性别为 Y 。每一只变色龙都有一个满足如下限制的初始颜色：

- X 性别的变色龙颜色两两不同。
- 对于每一只 X 性别的变色龙，恰有一只 Y 性别的变色龙颜色与之相同。

每只变色龙都喜欢另一只变色龙，且满足：

- 每只变色龙喜欢的必定是异性。
- 每只变色龙与其喜欢的变色龙具有不同的初始颜色。
- 不存在两只变色龙喜欢同一只变色龙。

现在你可以组织一些会议，在一次会议中：

- 若变色龙 a 喜欢的变色龙 b 参加了会议，则 a 的皮肤颜色为 b 的初始颜色。
- 否则， a 的皮肤颜色为 a 的初始颜色。

你可以得知参加这次会议的变色龙的不同的皮肤数量。请在不超过 20000 次会议中，确定初始颜色相同的每一对变色龙。

注意：性别为 X 的变色龙编号不一定是 1 到 n 。

数据范围： $n \leq 500$ 。

Hint

Hint

Hint 1: 考虑什么时候询问 $\{x, y\}$ 的答案是 1。

Hint

Hint 1: 考虑什么时候询问 $\{x, y\}$ 的答案是 1。

Hint 2: 若询问 $\{x, y\}$ 的答案为 1, 则我们在 x, y 之间连边。
假设我们已知图的形态, 该如何求解答案。

Hint

Hint 1: 考虑什么时候询问 $\{x, y\}$ 的答案是 1。

Hint 2: 若询问 $\{x, y\}$ 的答案为 1, 则我们在 x, y 之间连边。

假设我们已知图的形态, 该如何求解答案。

Hint 3: 考虑求出图的形态。

Hint

Hint 1: 考虑什么时候询问 $\{x, y\}$ 的答案是 1。

Hint 2: 若询问 $\{x, y\}$ 的答案为 1, 则我们在 x, y 之间连边。

假设我们已知图的形态, 该如何求解答案。

Hint 3: 考虑求出图的形态。

Hint 4: 考虑增量构造。

Hint

Hint 1: 考虑什么时候询问 $\{x, y\}$ 的答案是 1。

Hint 2: 若询问 $\{x, y\}$ 的答案为 1, 则我们在 x, y 之间连边。

假设我们已知图的形态, 该如何求解答案。

Hint 3: 考虑求出图的形态。

Hint 4: 考虑增量构造。

Hint 5: 图是一个二分图。

Hint

Hint 1: 考虑什么时候询问 $\{x, y\}$ 的答案是 1。

Hint 2: 若询问 $\{x, y\}$ 的答案为 1, 则我们在 x, y 之间连边。

假设我们已知图的形态, 该如何求解答案。

Hint 3: 考虑求出图的形态。

Hint 4: 考虑增量构造。

Hint 5: 图是一个二分图。

Hint 6: 对两侧分别二分。

Solution

Solution

我们询问 $\{a, b\}$, 则答案是 1 当且仅当满足以下三者之一:

Solution

我们询问 $\{a, b\}$, 则答案是 1 当且仅当满足以下三者之一:

- a, b 的颜色相同。

Solution

我们询问 $\{a, b\}$, 则答案是 1 当且仅当满足以下三者之一:

- a, b 的颜色相同。
- a 单向喜欢 b 。

Solution

我们询问 $\{a, b\}$, 则答案是 1 当且仅当满足以下三者之一:

- a, b 的颜色相同。
- a 单向喜欢 b 。
- b 单向喜欢 a 。

Solution

我们询问 $\{a, b\}$, 则答案是 1 当且仅当满足以下三者之一:

- a, b 的颜色相同。
- a 单向喜欢 b 。
- b 单向喜欢 a 。

我们在这样的 (a, b) 之间连边, 则一个点的度数要么是 1, 要么是 3。

Solution

我们询问 $\{a, b\}$, 则答案是 1 当且仅当满足以下三者之一:

- a, b 的颜色相同。
- a 单向喜欢 b 。
- b 单向喜欢 a 。

我们在这样的 (a, b) 之间连边, 则一个点的度数要么是 1, 要么是 3。

若一个点 x 的度数是 1, 设与其相连的点为 y , 则 x, y 必定颜色相同。

Solution

我们询问 $\{a, b\}$, 则答案是 1 当且仅当满足以下三者之一:

- a, b 的颜色相同。
- a 单向喜欢 b 。
- b 单向喜欢 a 。

我们在这样的 (a, b) 之间连边, 则一个点的度数要么是 1, 要么是 3。

若一个点 x 的度数是 1, 设与其相连的点为 y , 则 x, y 必定颜色相同。

若一个点 x 的度数是 3, 则设其与其相连的点为 a, b, c 。

Solution

我们询问 $\{a, b\}$, 则答案是 1 当且仅当满足以下三者之一:

- a, b 的颜色相同。
- a 单向喜欢 b 。
- b 单向喜欢 a 。

我们在这样的 (a, b) 之间连边, 则一个点的度数要么是 1, 要么是 3。

若一个点 x 的度数是 1, 设与其相连的点为 y , 则 x, y 必定颜色相同。

若一个点 x 的度数是 3, 则设其与其相连的点为 a, b, c 。

我们询问 $\{x, a, b\}$, 若答案为 1, 则 x 一定喜欢 c 。

Solution

我们询问 $\{a, b\}$, 则答案是 1 当且仅当满足以下三者之一:

- a, b 的颜色相同。
- a 单向喜欢 b 。
- b 单向喜欢 a 。

我们在这样的 (a, b) 之间连边, 则一个点的度数要么是 1, 要么是 3。

若一个点 x 的度数是 1, 设与其相连的点为 y , 则 x, y 必定颜色相同。

若一个点 x 的度数是 3, 则设其与其相连的点为 a, b, c 。

我们询问 $\{x, a, b\}$, 若答案为 1, 则 x 一定喜欢 c 。

因此我们可以求出哪些边是喜欢关系, 剩下的边连接的就是颜色相同的两只变色龙了。

Solution

我们询问 $\{a, b\}$, 则答案是 1 当且仅当满足以下三者之一:

- a, b 的颜色相同。
- a 单向喜欢 b 。
- b 单向喜欢 a 。

我们在这样的 (a, b) 之间连边, 则一个点的度数要么是 1, 要么是 3。

若一个点 x 的度数是 1, 设与其相连的点为 y , 则 x, y 必定颜色相同。

若一个点 x 的度数是 3, 则设其与其相连的点为 a, b, c 。

我们询问 $\{x, a, b\}$, 若答案为 1, 则 x 一定喜欢 c 。

因此我们可以求出哪些边是喜欢关系, 剩下的边连接的就是颜色相同的两只变色龙了。

考虑求出所有的边, 再通过上面的方式求出答案。

Solution

Solution

设 S 是一个独立集, 则 $S \cup \{x\}$ 的答案是 $|S| + 1$, 当且仅当 $S \cup \{x\}$ 是独立集。

Solution

设 S 是一个独立集，则 $S \cup \{x\}$ 的答案是 $|S| + 1$ ，当且仅当 $S \cup \{x\}$ 是独立集。

因此，我们可以通过二分的方式，求出 x 与一个独立集的所有连边。

Solution

设 S 是一个独立集，则 $S \cup \{x\}$ 的答案是 $|S| + 1$ ，当且仅当 $S \cup \{x\}$ 是独立集。

因此，我们可以通过二分的方式，求出 x 与一个独立集的所有连边。

考虑增量构造，每次加入一个点，并维护图的形态。

Solution

设 S 是一个独立集，则 $S \cup \{x\}$ 的答案是 $|S| + 1$ ，当且仅当 $S \cup \{x\}$ 是独立集。

因此，我们可以通过二分的方式，求出 x 与一个独立集的所有连边。

考虑增量构造，每次加入一个点，并维护图的形态。

因为图必定是二分图，所以我们可以把它划分成两个独立集，然后分别求出当前点与两个独立集之间的连边。

Solution

设 S 是一个独立集，则 $S \cup \{x\}$ 的答案是 $|S| + 1$ ，当且仅当 $S \cup \{x\}$ 是独立集。

因此，我们可以通过二分的方式，求出 x 与一个独立集的所有连边。

考虑增量构造，每次加入一个点，并维护图的形态。

因为图必定是二分图，所以我们可以把它划分成两个独立集，然后分别求出当前点与两个独立集之间的连边。

交互次数为 $\mathcal{O}(n \log n)$ 。

[COTS 2022] 帽子

[COTS 2022] 帽子

有 n 个人，每个人头上戴着一个黑色或白色的帽子。每个人能看到除了自己外所有人的帽子颜色，且会根据别人的帽子颜色来猜测自己的帽子颜色。

你需要帮他们构造一种猜测策略，使得对于所有 2^n 种情况，都满足：

- 设有 c_1 个人戴了黑色帽子，则其中至少有 $\lfloor \frac{c_1}{2} \rfloor$ 人猜对自己帽子的颜色。
- 设有 c_2 个人戴了白色帽子，则其中至少有 $\lfloor \frac{c_2}{2} \rfloor$ 人猜对自己帽子的颜色。

数据范围： $n \leq 18$ 。

Hint

Hint

Hint 1: 考虑建图。

Hint

Hint 1: 考虑建图。

Hint 2: 考虑构造一个 2^n 个点的图，且每个点的度数均为 n 。

Hint

Hint 1: 考虑建图。

Hint 2: 考虑构造一个 2^n 个点的图，且每个点的度数均为 n 。

Hint 3: 考虑构造欧拉回路。

Hint

Hint 1: 考虑建图。

Hint 2: 考虑构造一个 2^n 个点的图，且每个点的度数均为 n 。

Hint 3: 考虑构造欧拉回路。

Hint 4: 考虑拆点。

Solution

Solution

先考虑如何让至少 $\lfloor \frac{n}{2} \rfloor$ 人猜对自己帽子的颜色。

Solution

先考虑如何让至少 $\lfloor \frac{n}{2} \rfloor$ 人猜对自己帽子的颜色。

设白色为 0，黑色为 1，考虑构造一个 2^n 个点的图，
 $\forall 0 \leq i < n, i \notin S$ ，在 S 和 $S \cup \{i\}$ 之间连一条边，并对整张图定向。

Solution

先考虑如何让至少 $\lfloor \frac{n}{2} \rfloor$ 人猜对自己帽子的颜色。

设白色为 0，黑色为 1，考虑构造一个 2^n 个点的图，
 $\forall 0 \leq i < n, i \notin S$ ，在 S 和 $S \cup \{i\}$ 之间连一条边，并对整张图定向。

若最终边的方向是 $S \rightarrow S \cup \{i\}$ ，则第 i 个人在 S 状态下猜测自己帽子的颜色为黑色；若最终边的方向是 $S \cup \{i\} \rightarrow S$ ，则第 i 个人在 S 状态（第 i 个人看不到自己帽子的颜色，所以默认第 i 位为 0）下猜测自己帽子的颜色为白色。

Solution

Solution

那么对于一个状态 S ，每一条入边对应一个猜对的人，每一条出边对应一个猜错的人。

Solution

那么对于一个状态 S ，每一条入边对应一个猜对的人，每一条出边对应一个猜错的人。

我们建立一个虚点连向所有度数为奇数的人，然后跑欧拉回路，则每个点在原图中的出入度最多差 1，因此至少有 $\lfloor \frac{n}{2} \rfloor$ 人能猜对。

Solution

那么对于一个状态 S ，每一条入边对应一个猜对的人，每一条出边对应一个猜错的人。

我们建立一个虚点连向所有度数为奇数的人，然后跑欧拉回路，则每个点在原图中的出入度最多差 1，因此至少有 $\lfloor \frac{n}{2} \rfloor$ 人能猜对。

再考虑如何让黑白分别成立，我们把每个点 S 拆成 $(S, 0)$ 和 $(S, 1)$ 两个点，然后在 $(S, 0)$ 和 $(S \cup \{i\}, 1)$ 之间连边即可。

[集训队互测 2023-2024] Permutation Counting 2

[集训队互测 2023-2024] Permutation Counting 2

给定 n ，对每组 $x, y \in [0, n)$ ，求有多少个 $1 \sim n$ 的排列满足以下条件：

- $\sum_{i=1}^{n-1} [p_i < p_{i+1}] = x$ 。
- $\sum_{i=1}^{n-1} [p_i^{-1} < p_{i+1}^{-1}] = y$ 。

其中 p^{-1} 表示 p 的逆排列，答案对给定的质数取模。

数据范围： $n \leq 500$ 。

时间限制： 3 s。

Hint

Hint

Hint 1: 对恰好进行二项式反演。

Hint

Hint 1: 对恰好进行二项式反演。

Hint 2: 构造双射。

Solution

Solution

看到恰好可以条件反射想到二项式反演。

Solution

看到恰好可以条件反射想到二项式反演。

我们只需求出 $f_{i,j}$ 表示在原排列和逆排列中分别强制指定了 i, j 个连续上升段的方案数，就可以二维二项式反演得到答案。

Solution

看到恰好可以条件反射想到二项式反演。

我们只需求出 $f_{i,j}$ 表示在原排列和逆排列中分别强制指定了 i, j 个连续上升段的方案数，就可以二维二项式反演得到答案。

设 $c_{i,j}$ 表示 p^{-1} 中第 i 个连续上升段中的值，有多少与 p 中第 j 个连续上升段对应的位置区间 $[l_j, r_j]$ 包含。

Solution

看到恰好可以条件反射想到二项式反演。

我们只需求出 $f_{i,j}$ 表示在原排列和逆排列中分别强制指定了 i, j 个连续上升段的方案数，就可以二维二项式反演得到答案。

设 $c_{i,j}$ 表示 p^{-1} 中第 i 个连续上升段中的值，有多少与 p 中第 j 个连续上升段对应的位置区间 $[l_j, r_j]$ 包含。

则有结论： p 与矩阵 c 构成双射。证明如下：

Solution

看到恰好可以条件反射想到二项式反演。

我们只需求出 $f_{i,j}$ 表示在原排列和逆排列中分别强制指定了 i, j 个连续上升段的方案数，就可以二维二项式反演得到答案。

设 $c_{i,j}$ 表示 p^{-1} 中第 i 个连续上升段中的值，有多少与 p 中第 j 个连续上升段对应的位置区间 $[l_j, r_j]$ 包含。

则有结论： p 与矩阵 c 构成双射。证明如下：

考虑如何确定 p^{-1} 的第一个上升段中的数的值。

Solution

看到恰好可以条件反射想到二项式反演。

我们只需求出 $f_{i,j}$ 表示在原排列和逆排列中分别强制指定了 i, j 个连续上升段的方案数，就可以二维二项式反演得到答案。

设 $c_{i,j}$ 表示 p^{-1} 中第 i 个连续上升段中的值，有多少与 p 中第 j 个连续上升段对应的位置区间 $[l_j, r_j]$ 包含。

则有结论： p 与矩阵 c 构成双射。证明如下：

考虑如何确定 p^{-1} 的第一个上升段中的数的值。

设这些值构成的集合为 S ，则 $\{p_x \mid x \in S\}$ 必定是一个前缀（因为在 p^{-1} 中是位置上的一个前缀）。

Solution

看到恰好可以条件反射想到二项式反演。

我们只需求出 $f_{i,j}$ 表示在原排列和逆排列中分别强制指定了 i, j 个连续上升段的方案数，就可以二维二项式反演得到答案。

设 $c_{i,j}$ 表示 p^{-1} 中第 i 个连续上升段中的值，有多少与 p 中第 j 个连续上升段对应的位置区间 $[l_j, r_j]$ 包含。

则有结论： p 与矩阵 c 构成双射。证明如下：

考虑如何确定 p^{-1} 的第一个上升段中的数的值。

设这些值构成的集合为 S ，则 $\{p_x \mid x \in S\}$ 必定是一个前缀（因为在 p^{-1} 中是位置上的一个前缀）。

又因为 $c_{1,i} = |[l_i, r_i] \cap S|$ ，且 $[l_i, r_i]$ 是 p 中的一个上升段，所以 $[l_i, r_i] \cap S$ 必定对应 $[l_i, r_i]$ 的一个前缀。

Solution

看到恰好可以条件反射想到二项式反演。

我们只需求出 $f_{i,j}$ 表示在原排列和逆排列中分别强制指定了 i, j 个连续上升段的方案数，就可以二维二项式反演得到答案。

设 $c_{i,j}$ 表示 p^{-1} 中第 i 个连续上升段中的值，有多少与 p 中第 j 个连续上升段对应的位置区间 $[l_j, r_j]$ 包含。

则有结论： p 与矩阵 c 构成双射。证明如下：

考虑如何确定 p^{-1} 的第一个上升段中的数的值。

设这些值构成的集合为 S ，则 $\{p_x \mid x \in S\}$ 必定是一个前缀（因为在 p^{-1} 中是位置上的一个前缀）。

又因为 $c_{1,i} = |[l_i, r_i] \cap S|$ ，且 $[l_i, r_i]$ 是 p 中的一个上升段，所以 $[l_i, r_i] \cap S$ 必定对应 $[l_i, r_i]$ 的一个前缀。

因此， S 是唯一确定的。

Solution

看到恰好可以条件反射想到二项式反演。

我们只需求出 $f_{i,j}$ 表示在原排列和逆排列中分别强制指定了 i, j 个连续上升段的方案数，就可以二维二项式反演得到答案。

设 $c_{i,j}$ 表示 p^{-1} 中第 i 个连续上升段中的值，有多少与 p 中第 j 个连续上升段对应的位置区间 $[l_j, r_j]$ 包含。

则有结论： p 与矩阵 c 构成双射。证明如下：

考虑如何确定 p^{-1} 的第一个上升段中的数的值。

设这些值构成的集合为 S ，则 $\{p_x \mid x \in S\}$ 必定是一个前缀（因为在 p^{-1} 中是位置上的一个前缀）。

又因为 $c_{1,i} = |[l_i, r_i] \cap S|$ ，且 $[l_i, r_i]$ 是 p 中的一个上升段，所以 $[l_i, r_i] \cap S$ 必定对应 $[l_i, r_i]$ 的一个前缀。

因此， S 是唯一确定的。

删掉 p^{-1} 的第一个上升段和 p 中 S 对应的位置集合后，不难发现 p^{-1} 的第二个上升段及其对应的 S' 也是唯一确定的。

Solution

Solution

一直这样做下去，可知矩阵 c 能唯一确定 p 。

Solution

一直这样做下去，可知矩阵 c 能唯一确定 p 。

因此我们现在只需对任意 i, j ，计数满足以下条件的矩阵 c 的个数：

Solution

一直这样做下去，可知矩阵 c 能唯一确定 p 。

因此我们现在只需对任意 i, j ，计数满足以下条件的矩阵 c 的个数：

- c 的大小是 $i \times j$ 。

Solution

一直这样做下去，可知矩阵 c 能唯一确定 p 。

因此我们现在只需对任意 i, j ，计数满足以下条件的矩阵 c 的个数：

- c 的大小是 $i \times j$ 。
- c 中元素的和为 n 。

Solution

一直这样做下去，可知矩阵 c 能唯一确定 p 。

因此我们现在只需对任意 i, j ，计数满足以下条件的矩阵 c 的个数：

- c 的大小是 $i \times j$ 。
- c 中元素的和为 n 。
- c 的每行每列和为正数。

Solution

一直这样做下去，可知矩阵 c 能唯一确定 p 。

因此我们现在只需对任意 i, j ，计数满足以下条件的矩阵 c 的个数：

- c 的大小是 $i \times j$ 。
- c 中元素的和为 n 。
- c 的每行每列和为正数。

只需对第三个条件进行容斥即可。

Solution

一直这样做下去，可知矩阵 c 能唯一确定 p 。

因此我们现在只需对任意 i, j ，计数满足以下条件的矩阵 c 的个数：

- c 的大小是 $i \times j$ 。
- c 中元素的和为 n 。
- c 的每行每列和为正数。

只需对第三个条件进行容斥即可。
容易做到 $\mathcal{O}(n^3)$ 的时间复杂度。

[IOI2018] seats

[IOI2018] seats

给定一个 $n \times m$ 的网格，网格中每个位置有一个数，所有数构成一个 $0 \sim nm - 1$ 的排列。

q 次操作，每次操作交换两个元素，并查询有多少个 i ，满足 $0 \sim i - 1$ 在网格中构成的图形是一个长方形。

数据范围： $nm \leq 10^6, q \leq 5 \times 10^4$ 。

时间限制：3 s。

Hint

Hint

Hint 1: 考虑 i 什么时候合法。

Hint

Hint 1: 考虑 i 什么时候合法。

Hint 2: 若从 $(x_{\max} - x_{\min} + 1)(y_{\max} - y_{\min} + 1)$ 的角度入手, 那大概率没救了。

Hint

Hint 1: 考虑 i 什么时候合法。

Hint 2: 若从 $(x_{\max} - x_{\min} + 1)(y_{\max} - y_{\min} + 1)$ 的角度入手, 那大概率没救了。

Hint 3: 考虑一个必要条件: 对于所有 $\geq i$ 的数, 与其相邻的四个点中, 最多有一个 $< i$ 的数。

Hint

Hint 1: 考虑 i 什么时候合法。

Hint 2: 若从 $(x_{\max} - x_{\min} + 1)(y_{\max} - y_{\min} + 1)$ 的角度入手, 那大概率没救了。

Hint 3: 考虑一个必要条件: 对于所有 $\geq i$ 的数, 与其相邻的四个点中, 最多有一个 $< i$ 的数。

Hint 4: 考虑怎么样才能让这个条件变得充要。

Hint

Hint 1: 考虑 i 什么时候合法。

Hint 2: 若从 $(x_{\max} - x_{\min} + 1)(y_{\max} - y_{\min} + 1)$ 的角度入手, 那大概率没救了。

Hint 3: 考虑一个必要条件: 对于所有 $\geq i$ 的数, 与其相邻的四个点中, 最多有一个 $< i$ 的数。

Hint 4: 考虑怎么样才能让这个条件变得充要。

Hint 5: 考虑用线段树优化。

Solution

Solution

在满足「对于所有 $\geq i$ 的数，与其相邻的四个点中，最多有一个 $< i$ 的数」的前提下， $0 \sim i - 1$ 在网格中构成的图形只能是若干不交长方形的并。

Solution

在满足「对于所有 $\geq i$ 的数，与其相邻的四个点中，最多有一个 $< i$ 的数」的前提下， $0 \sim i-1$ 在网格中构成的图形只能是若干不交长方形的并。

我们需要限制长方形的数量为 1，可以统计有多少个 $< i$ 的数，使得其左侧和上侧的数都 $\geq i$ 。

Solution

在满足「对于所有 $\geq i$ 的数，与其相邻的四个点中，最多有一个 $< i$ 的数」的前提下， $0 \sim i-1$ 在网格中构成的图形只能是若干不交长方形的并。

我们需要限制长方形的数量为 1，可以统计有多少个 $< i$ 的数，使得其左侧和上侧的数都 $\geq i$ 。

设 f_i 表示有多少 $\geq i$ 的数满足，与其相邻的四个点中，有超过一个 $< i$ 的数。

Solution

在满足「对于所有 $\geq i$ 的数，与其相邻的四个点中，最多有一个 $< i$ 的数」的前提下， $0 \sim i-1$ 在网格中构成的图形只能是若干不交长方形的并。

我们需要限制长方形的数量为 1，可以统计有多少个 $< i$ 的数，使得其左侧和上侧的数都 $\geq i$ 。

设 f_i 表示有多少 $\geq i$ 的数满足，与其相邻的四个点中，有超过一个 $< i$ 的数。

设 g_i 表示有多少 $< i$ 的数满足其左侧和上侧的数都 $\geq i$ 。

Solution

在满足「对于所有 $\geq i$ 的数，与其相邻的四个点中，最多有一个 $< i$ 的数」的前提下， $0 \sim i-1$ 在网格中构成的图形只能是若干不交长方形的并。

我们需要限制长方形的数量为 1，可以统计有多少个 $< i$ 的数，使得其左侧和上侧的数都 $\geq i$ 。

设 f_i 表示有多少 $\geq i$ 的数满足，与其相邻的四个点中，有超过一个 $< i$ 的数。

设 g_i 表示有多少 $< i$ 的数满足其左侧和上侧的数都 $\geq i$ 。

显然 $f_i + g_i \geq 1$ ，而我们需要统计的是 $f_i + g_i = 1$ 的 i 的数量。

Solution

在满足「对于所有 $\geq i$ 的数，与其相邻的四个点中，最多有一个 $< i$ 的数」的前提下， $0 \sim i-1$ 在网格中构成的图形只能是若干不交长方形的并。

我们需要限制长方形的数量为 1，可以统计有多少个 $< i$ 的数，使得其左侧和上侧的数都 $\geq i$ 。

设 f_i 表示有多少 $\geq i$ 的数满足，与其相邻的四个点中，有超过一个 $< i$ 的数。

设 g_i 表示有多少 $< i$ 的数满足其左侧和上侧的数都 $\geq i$ 。

显然 $f_i + g_i \geq 1$ ，而我们需要统计的是 $f_i + g_i = 1$ 的 i 的数量。

使用线段树维护区间最小值及其数量即可，时间复杂度 $\mathcal{O}(nm + q \log nm)$ 。

卡牌

卡牌

给定 m 个四元组 (a_i, b_i, c_i, d_i) , 保证 $1 \leq a_i, b_i, c_i, d_i \leq n$ 。

求有多少四元组 (A, B, C, D) , 满足

$$\forall i, [A > a_i] + [B > b_i] + [C > c_i] + [D > d_i] \geq 3。$$

数据范围: $n, m \leq 5 \times 10^5$ 。

时间限制: 1 s。

Hint

Hint

Hint 1: 将问题转化为带权的三维情况。

Hint

Hint 1: 将问题转化为带权的三维情况。

Hint 2: 考虑三维的情况怎么做。

Solution

Solution

相当于对于任意两维（以 a, b 为例）不能同时有
 $A \leq a_i, B \leq b_i$ 。

Solution

相当于对于任意两维（以 a, b 为例）不能同时有

$$A \leq a_i, B \leq b_i。$$

我们可以把限制描述成 $A \leq a_i \Rightarrow B > b_i$ 或

$B \leq b_i \Rightarrow A > a_i$ ，可以发现这两个限制只需保留一个，我们考虑只保留高维对低维的限制（设 A, B, C, D 分别对应第一、二、三、四维）。

Solution

相当于对于任意两维（以 a, b 为例）不能同时有

$$A \leq a_i, B \leq b_i。$$

我们可以把限制描述成 $A \leq a_i \Rightarrow B > b_i$ 或

$B \leq b_i \Rightarrow A > a_i$ ，可以发现这两个限制只需保留一个，我们考虑只保留高维对低维的限制（设 A, B, C, D 分别对应第一、二、三、四维）。

考虑从大到小枚举 D ，则此时 A, B, C 会有一个对应的下界限制，且这个限制会不断变紧，每次相当于删除 A, B, C 的一个前缀。

Solution

相当于对于任意两维（以 a, b 为例）不能同时有

$$A \leq a_i, B \leq b_i。$$

我们可以把限制描述成 $A \leq a_i \Rightarrow B > b_i$ 或

$B \leq b_i \Rightarrow A > a_i$ ，可以发现这两个限制只需保留一个，我们考虑只保留高维对低维的限制（设 A, B, C, D 分别对应第一、二、三、四维）。

考虑从大到小枚举 D ，则此时 A, B, C 会有一个对应的下界限制，且这个限制会不断变紧，每次相当于删除 A, B, C 的一个前缀。

设 a, b, c 这三维每个位置被删除的时间为 t_a, t_b, t_c ，则对于一个合法的三元组 (A, B, C) ，它对答案的贡献就是 $\min(t_a(A), t_b(B), t_c(C))$ 。

Solution

相当于对于任意两维（以 a, b 为例）不能同时有

$$A \leq a_i, B \leq b_i。$$

我们可以把限制描述成 $A \leq a_i \Rightarrow B > b_i$ 或

$B \leq b_i \Rightarrow A > a_i$ ，可以发现这两个限制只需保留一个，我们考虑只保留高维对低维的限制（设 A, B, C, D 分别对应第一、二、三、四维）。

考虑从大到小枚举 D ，则此时 A, B, C 会有一个对应的下界限制，且这个限制会不断变紧，每次相当于删除 A, B, C 的一个前缀。

设 a, b, c 这三维每个位置被删除的时间为 t_a, t_b, t_c ，则对于一个合法的三元组 (A, B, C) ，它对答案的贡献就是 $\min(t_a(A), t_b(B), t_c(C))$ 。

现在我们将问题转化为了一个带权的三维问题，考虑三维的情况怎么做。

Solution

Solution

考虑枚举第三维，那么对第一、二维会有一个下界限制，而第二维的值对第一维的值同样是一个下界的限制，因此我们只需用双指针找到三 $\rightarrow$ 一的限制和二 $\rightarrow$ 一的限制大小关系变化的分界线，然后查询前缀和即可。

Solution

考虑枚举第三维，那么对第一、二维会有一个下界限制，而第二维的值对第一维的值同样是一个下界的限制，因此我们只需用双指针找到三 $\rightarrow$ 一的限制和二 $\rightarrow$ 一的限制大小关系变化的分界线，然后查询前缀和即可。

于是对于四维的情况，我们只需枚举这个 $\min$ 是哪一维，然后把它当作上面的第三维做即可。

Solution

考虑枚举第三维，那么对第一、二维会有一个下界限制，而第二维的值对第一维的值同样是一个下界的限制，因此我们只需用双指针找到三 $\rightarrow$ 一的限制和二 $\rightarrow$ 一的限制大小关系变化的分界线，然后查询前缀和即可。

于是对于四维的情况，我们只需枚举这个 $\min$ 是哪一维，然后把它当作上面的第三维做即可。

时间复杂度 $\mathcal{O}(n)$ 。

[NOI2018] 情报中心

[NOI2018] 情报中心

给定一棵 n 个点的树，边有正边权。

给定树上的 m 条路径，每条路径有一个费用。

要求选出两条至少有一条公共边的路径，使得它们的并的边权和减去它们的费用之和最大。

数据范围：

$$n \leq 5 \times 10^4, m \leq 10^5, \sum n \leq 10^6, \sum m \leq 2 \times 10^6.$$

时间限制：8 s。

Hint

Hint

Hint 1: 考虑把并的边权和转化为四对点的距离之和。

Hint

Hint 1: 考虑把并的边权和转化为四对点的距离之和。

Hint 2: 考虑枚举 LCA。

Hint

Hint 1: 考虑把并的边权和转化为四对点的距离之和。

Hint 2: 考虑枚举 LCA。

Hint 3: 考虑把问题转化为求树的直径。

Hint

Hint 1: 考虑把并的边权和转化为四对点的距离之和。

Hint 2: 考虑枚举 LCA。

Hint 3: 考虑把问题转化为求树的直径。

Hint 4: 用线段树合并维护树的直径。

Solution

Solution

对于两条相交路径 $x_i \rightarrow y_i, x_j \rightarrow y_j$, 我们设
 $x_i \rightarrow x_j, y_i \rightarrow y_j$ 两条路径不相交, 则答案为:

Solution

对于两条相交路径 $x_i \rightarrow y_i, x_j \rightarrow y_j$, 我们设
 $x_i \rightarrow x_j, y_i \rightarrow y_j$ 两条路径不相交, 则答案为:

$$\frac{1}{2}(\text{dis}(x_i, y_i) + \text{dis}(x_j, y_j) + \text{dis}(x_i, x_j) + \text{dis}(y_i, y_j)) - v_i - v_j$$

Solution

对于两条相交路径 $x_i \rightarrow y_i, x_j \rightarrow y_j$, 我们设
 $x_i \rightarrow x_j, y_i \rightarrow y_j$ 两条路径不相交, 则答案为:

$$\frac{1}{2}(\text{dis}(x_i, y_i) + \text{dis}(x_j, y_j) + \text{dis}(x_i, x_j) + \text{dis}(y_i, y_j)) - v_i - v_j$$

考虑枚举 $l = \text{lca}(x_i, x_j)$, 则我们只需最大化
 $\text{dep}_{x_i} + \text{dep}_{x_j} + \text{dis}(y_i, y_j) + \text{len}_i + \text{len}_j - 2v_i - 2v_j$ 。

Solution

对于两条相交路径 $x_i \rightarrow y_i, x_j \rightarrow y_j$, 我们设 $x_i \rightarrow x_j, y_i \rightarrow y_j$ 两条路径不相交, 则答案为:

$$\frac{1}{2}(\text{dis}(x_i, y_i) + \text{dis}(x_j, y_j) + \text{dis}(x_i, x_j) + \text{dis}(y_i, y_j)) - v_i - v_j$$

考虑枚举 $l = \text{lca}(x_i, x_j)$, 则我们只需最大化 $\text{dep}_{x_i} + \text{dep}_{x_j} + \text{dis}(y_i, y_j) + \text{len}_i + \text{len}_j - 2v_i - 2v_j$ 。

令 $w_i = \text{len}_i - 2v_i + \text{dep}_{x_i}$, 则只需最大化 $\text{dis}(y_i, y_j) + w_i + w_j$ 。

Solution

对于两条相交路径 $x_i \rightarrow y_i, x_j \rightarrow y_j$, 我们设
 $x_i \rightarrow x_j, y_i \rightarrow y_j$ 两条路径不相交, 则答案为:

$$\frac{1}{2}(\text{dis}(x_i, y_i) + \text{dis}(x_j, y_j) + \text{dis}(x_i, x_j) + \text{dis}(y_i, y_j)) - v_i - v_j$$

考虑枚举 $l = \text{lca}(x_i, x_j)$, 则我们只需最大化
 $\text{dep}_{x_i} + \text{dep}_{x_j} + \text{dis}(y_i, y_j) + \text{len}_i + \text{len}_j - 2v_i - 2v_j$ 。

令 $w_i = \text{len}_i - 2v_i + \text{dep}_{x_i}$, 则只需最大化
 $\text{dis}(y_i, y_j) + w_i + w_j$ 。

注意到把 $w_i + \infty$ 挂在 y_i 上, 则问题等价于求树的直径。

Solution

对于两条相交路径 $x_i \rightarrow y_i, x_j \rightarrow y_j$, 我们设
 $x_i \rightarrow x_j, y_i \rightarrow y_j$ 两条路径不相交, 则答案为:

$$\frac{1}{2}(\text{dis}(x_i, y_i) + \text{dis}(x_j, y_j) + \text{dis}(x_i, x_j) + \text{dis}(y_i, y_j)) - v_i - v_j$$

考虑枚举 $l = \text{lca}(x_i, x_j)$, 则我们只需最大化
 $\text{dep}_{x_i} + \text{dep}_{x_j} + \text{dis}(y_i, y_j) + \text{len}_i + \text{len}_j - 2v_i - 2v_j$ 。

令 $w_i = \text{len}_i - 2v_i + \text{dep}_{x_i}$, 则只需最大化
 $\text{dis}(y_i, y_j) + w_i + w_j$ 。

注意到把 $w_i + \infty$ 挂在 y_i 上, 则问题等价于求树的直径。

对于每个 i , 把 l 的范围限定在 x_i 到 $\text{lca}(x_i, y_i)$ 的倒数第二个点, 这样就可以做到统计的不重不漏。

Solution

对于两条相交路径 $x_i \rightarrow y_i, x_j \rightarrow y_j$, 我们设 $x_i \rightarrow x_j, y_i \rightarrow y_j$ 两条路径不相交, 则答案为:

$$\frac{1}{2}(\text{dis}(x_i, y_i) + \text{dis}(x_j, y_j) + \text{dis}(x_i, x_j) + \text{dis}(y_i, y_j)) - v_i - v_j$$

考虑枚举 $l = \text{lca}(x_i, x_j)$, 则我们只需最大化 $\text{dep}_{x_i} + \text{dep}_{x_j} + \text{dis}(y_i, y_j) + \text{len}_i + \text{len}_j - 2v_i - 2v_j$ 。

令 $w_i = \text{len}_i - 2v_i + \text{dep}_{x_i}$, 则只需最大化 $\text{dis}(y_i, y_j) + w_i + w_j$ 。

注意到把 $w_i + \infty$ 挂在 y_i 上, 则问题等价于求树的直径。

对于每个 i , 把 l 的范围限定在 x_i 到 $\text{lca}(x_i, y_i)$ 的倒数第二个点, 这样就可以做到统计的不重不漏。

使用 $\mathcal{O}(1)$ -LCA 和线段树合并维护即可, 设 n, m 同阶, 时间复杂度 $\mathcal{O}(n \log n)$ 。

[集训队互测 2024-2025] 线段树与区间加

[集训队互测 2024-2025] 线段树与区间加

给定一棵 $[1, n]$ 的线段树（每次不一定从中点分治），要求支持：

- 在该线段树上执行区间加法操作。
- 查询 $\sum_u va_u \times sum_u + \sum_u vb_u \times tag_u$ 。
其中 sum_u 表示节点 u 对应区间的元素和（未考虑祖先的 tag ）， tag_u 表示节点 u 未下传的加法标记。

数据范围： $n, q \leq 2 \times 10^5$ 。

时间限制： 6 s。

Hint

Hint

Hint 1: 考虑把 sum 的贡献转化到 tag 上。

Hint

Hint 1: 考虑把 sum 的贡献转化到 tag 上。

Hint 2: 考虑对 tag 做前缀和。

Hint

Hint 1: 考虑把 sum 的贡献转化到 tag 上。

Hint 2: 考虑对 tag 做前缀和。

Hint 3: 考虑魔改重链剖分。

Solution

Solution

首先 sum_u 等于对应区间的实际元素和，减去 u 祖先的 tag 之和乘上 u 对应区间的长度 len_u 。

Solution

首先 sum_u 等于对应区间的实际元素和，减去 u 祖先的 tag 之和乘上 u 对应区间的长度 len_u 。

因此我们可以对每个点 u ，将其对应区间的每个点 w 加上 va_u ，并将其所有祖先的 vb 减去 $va_u \times len_u$ 。

Solution

首先 sum_u 等于对应区间的实际元素和，减去 u 祖先的 tag 之和乘上 u 对应区间的长度 len_u 。

因此我们可以对每个点 u ，将其对应区间的每个点 w 加上 va_u ，并将其所有祖先的 vb 减去 $va_u \times len_u$ 。

最后答案就是 $\sum_{i=1}^n w_i \times a_i + \sum_u vb_u \times tag_u$ ，前者的处理是平凡的。于是我们只需考虑怎么快速维护 $\sum_u vb_u \times tag_u$ 。

Solution

首先 sum_u 等于对应区间的实际元素和，减去 u 祖先的 tag 之和乘上 u 对应区间的长度 len_u 。

因此我们可以对每个点 u ，将其对应区间的每个点 w 加上 va_u ，并将其所有祖先的 vb 减去 $va_u \times len_u$ 。

最后答案就是 $\sum_{i=1}^n w_i \times a_i + \sum_u vb_u \times tag_u$ ，前者的处理是平凡的。于是我们只需考虑怎么快速维护 $\sum_u vb_u \times tag_u$ 。

设区间 $[x, x]$ 在线段树上对应的叶子是 p_x ，则对 $[l, r]$ 执行区间加，相当于先把 $rt \rightarrow p_l$ 和 $rt \rightarrow p_r$ 这两条链 pushdown，然后将 $LCA(p_l, p_r) \rightarrow p_l$ 的所有右儿子，以及 $LCA(p_l, p_r) \rightarrow p_r$ 的所有左儿子的 tag 加上输入的权值。

Solution

首先 sum_u 等于对应区间的实际元素和，减去 u 祖先的 tag 之和乘上 u 对应区间的长度 len_u 。

因此我们可以对每个点 u ，将其对应区间的每个点 w 加上 va_u ，并将其所有祖先的 vb 减去 $va_u \times len_u$ 。

最后答案就是 $\sum_{i=1}^n w_i \times a_i + \sum_u vb_u \times tag_u$ ，前者的处理是平凡的。于是我们只需考虑怎么快速维护 $\sum_u vb_u \times tag_u$ 。

设区间 $[x, x]$ 在线段树上对应的叶子是 p_x ，则对 $[l, r]$ 执行区间加，相当于先把 $rt \rightarrow p_l$ 和 $rt \rightarrow p_r$ 这两条链 pushdown，然后将 $LCA(p_l, p_r) \rightarrow p_l$ 的所有右儿子，以及 $LCA(p_l, p_r) \rightarrow p_r$ 的所有左儿子的 tag 加上输入的权值。

这两个操作都比较棘手，考虑进行一些转化。

Solution

Solution

令 tag'_u 表示从 u 到根路径上所有点的 tag 的和, 则操作转化为将 $rt \rightarrow p_l$ 和 $rt \rightarrow p_r$ 这两条链的 tag' 清零, 并将 $LCA(p_l, p_r) \rightarrow p_l$ 的所有右儿子的子树, 以及 $LCA(p_l, p_r) \rightarrow p_r$ 的所有左儿子的子树内的 tag' 都加上输入的权值。答案变为 $\sum_u vb'_u \times tag'_u$, 其中 $vb'_u = vb_u - vb_{lson(u)} - vb_{rson(u)}$ 。

Solution

令 tag'_u 表示从 u 到根路径上所有点的 tag 的和, 则操作转化为将 $rt \rightarrow p_l$ 和 $rt \rightarrow p_r$ 这两条链的 tag' 清零, 并将 $LCA(p_l, p_r) \rightarrow p_l$ 的所有右儿子的子树, 以及 $LCA(p_l, p_r) \rightarrow p_r$ 的所有左儿子的子树内的 tag' 都加上输入的权值。答案变为 $\sum_u vb'_u \times tag'_u$, 其中 $vb'_u = vb_u - vb_{lson(u)} - vb_{rson(u)}$ 。

清零操作是容易处理的, 对于后者, 我们可以魔改一下重链剖分的顺序。

Solution

令 tag'_u 表示从 u 到根路径上所有点的 tag 的和, 则操作转化为将 $rt \rightarrow p_l$ 和 $rt \rightarrow p_r$ 这两条链的 tag' 清零, 并将 $LCA(p_l, p_r) \rightarrow p_l$ 的所有右儿子的子树, 以及 $LCA(p_l, p_r) \rightarrow p_r$ 的所有左儿子的子树内的 tag' 都加上输入的权值。答案变为 $\sum_u vb'_u \times tag'_u$, 其中 $vb'_u = vb_u - vb_{lson(u)} - vb_{rson(u)}$ 。

清零操作是容易处理的, 对于后者, 我们可以魔改一下重链剖分的顺序。

具体地, 处理到一条重链时, 我们先按顺序加入该重链上的所有点, 然后按顺序依次处理重链上的所有左儿子, 再按顺序处理重链上的所有右儿子。这样, 对于一条重链, 其所有左儿子子树和所有右儿子子树分别构成了连续的区间。

Solution

令 tag'_u 表示从 u 到根路径上所有点的 tag 的和, 则操作转化为将 $rt \rightarrow p_l$ 和 $rt \rightarrow p_r$ 这两条链的 tag' 清零, 并将 $LCA(p_l, p_r) \rightarrow p_l$ 的所有右儿子的子树, 以及 $LCA(p_l, p_r) \rightarrow p_r$ 的所有左儿子的子树内的 tag' 都加上输入的权值。答案变为 $\sum_u vb'_u \times tag'_u$, 其中 $vb'_u = vb_u - vb_{lson(u)} - vb_{rson(u)}$ 。

清零操作是容易处理的, 对于后者, 我们可以魔改一下重链剖分的顺序。

具体地, 处理到一条重链时, 我们先按顺序加入该重链上的所有点, 然后按顺序依次处理重链上的所有左儿子, 再按顺序处理重链上的所有右儿子。这样, 对于一条重链, 其所有左儿子子树和所有右儿子子树分别构成了连续的区间。

配合线段树, 即可做到 $\mathcal{O}(n + q \log^2 n)$ 的时间复杂度。

Solution

令 tag'_u 表示从 u 到根路径上所有点的 tag 的和, 则操作转化为将 $rt \rightarrow p_l$ 和 $rt \rightarrow p_r$ 这两条链的 tag' 清零, 并将 $LCA(p_l, p_r) \rightarrow p_l$ 的所有右儿子的子树, 以及 $LCA(p_l, p_r) \rightarrow p_r$ 的所有左儿子的子树内的 tag' 都加上输入的权值。答案变为 $\sum_u vb'_u \times tag'_u$, 其中 $vb'_u = vb_u - vb_{lson(u)} - vb_{rson(u)}$ 。

清零操作是容易处理的, 对于后者, 我们可以魔改一下重链剖分的顺序。

具体地, 处理到一条重链时, 我们先按顺序加入该重链上的所有点, 然后按顺序依次处理重链上的所有左儿子, 再按顺序处理重链上的所有右儿子。这样, 对于一条重链, 其所有左儿子子树和所有右儿子子树分别构成了连续的区间。

配合线段树, 即可做到 $\mathcal{O}(n + q \log^2 n)$ 的时间复杂度。

事实上, 可以用全局平衡二叉树做到 $\mathcal{O}(n + q \log n)$ 的时间复杂度。可以参见出题人的题解。

谢谢大家。

谢谢大家。
祝大家在 NOI2026 中取得好成绩！